

# Admissions Oracle — Change Log

**Original:** RedditUniAdmissionGame (prototype) **Updated:** admission-oracle (current version)

---

## 1. New Files Added

| File                                   | Purpose                                                                                     |
|----------------------------------------|---------------------------------------------------------------------------------------------|
| <code>server.js</code>                 | Express backend server — serves profiles, handles scores and leaderboard via REST API       |
| <code>package.json</code>              | Node project config — lists all dependencies (express, lowdb, bcryptjs, jsonwebtoken, etc.) |
| <code>README.md</code>                 | Setup and deployment documentation                                                          |
| <code>.env.example</code>              | Template for environment variables (API keys, port)                                         |
| <code>public/auth.jsx</code>           | Login and Register UI component                                                             |
| <code>public/index.html</code>         | Renamed from "Admissions Oracle.html" — required for server to serve it at <code>/</code>   |
| <code>scripts/scrape.js</code>         | Reddit scraper — fetches posts from r/collegeresults, extracts profiles via OpenRouter AI   |
| <code>scripts/download-logos.js</code> | Downloads school favicons from Google's favicon service into <code>public/logos/</code>     |
| <code>data/profiles.jsonl</code>       | 15 real applicant profiles converted from Reddit/RedNote posts                              |
| <code>data/game.json</code>            | Live database — stores user accounts and scores (auto-created on first run)                 |
| <code>public/logos/*.png</code>        | 90+ school logo images downloaded locally                                                   |

## 2. app.jsx — Major Rewrite

### Authentication system (entirely new)

- Added login/register flow using `auth.jsx` — users must create an account before playing
- JWT tokens stored in `localStorage` — session persists across browser refreshes
- Token validated against server on page load via `GET /api/me`
- Logout button clears token and resets all game state

### Profile loading — from server instead of inline data

- **Before:** profiles loaded from `window.PROFILES` (hardcoded in `data.js`)
- **After:** profiles fetched from `GET /api/profiles` on login; falls back to `window.PROFILES` if server is unreachable

### Full profile fetch before Phase 3

- Added `fullProfile` state variable
- When player locks their tier guess (end of Phase 2), app fetches the complete profile including `application_results` from `GET /api/profiles/:id`
- Fixes crash where Phase 3 tried to read school results that were stripped from the initial profile list
- Phase 3 and Phase 4 both use `fullProfile` instead of the stripped version

### Score submission — to server instead of local only

- `commitScore()` now POSTs to `POST /api/scores` with JWT auth header
- Server stores best score per user per profile
- Leaderboard button added to topbar

### Leaderboard screen (new)

- New `LeaderboardScreen` component showing global rankings
  - Displays: rank number, username, games played, total points
  - Your own row highlighted
  - Live stats: total profiles, total players
-

### 3. phase3-school.jsx — School Logos Added

#### SchoolLogo component (new)

- Each school card now shows the school's favicon loaded from `public/logos/[name].png`
- `getLogoFile(name)` maps 100+ school name variations to their local filename
- Falls back to a colored initial letter if the logo file is missing
- Fallback colors are brand-accurate (Harvard crimson, Yale blue, Stanford red, etc.)

#### SchoolCard — spring animation

- Cards animate with a subtle scale bounce (`scale(0.95) → scale(1)`) when tapped/clicked
- 

### 4. phase4-results.jsx — Celebration Animations Added

#### CSS animations injected at runtime (new)

- `ao-pop-in` — spring scale-up animation for correct rows
- `ao-shake-no` — side-to-side shake for wrong rows
- `ao-slide-up` — fade-up entrance for skipped rows
- `ao-confetti-fall` — particle animation for confetti bursts
- `ao-celebrate-banner` — pop-in for the celebration banner

#### ConfettiBurst component (new)

- Fires 14 colored confetti particles in a circular burst from each correct school
- Colors: green, blue, amber, pink, purple, cyan
- Particles animate outward and fade over 0.7s

#### CelebrationBanner component (new)

- Appears above score cards when accuracy  $\geq 50\%$  or score  $\geq 15$
- Shows 🎯 "Sharp eye!" for accuracy  $\geq 80\%$ , 🎉 "Nice work!" otherwise
- Animated pop-in entrance

#### ResultRow — updated

- Correct rows: `ao-correct-row` animation + confetti burst trigger
- Wrong rows: `ao-shake-no` animation

- School logos now appear inline next to school names in the results list
- 

## 5. Backend Server (server.js) — New

### User accounts

- `POST /api/register` — creates account, hashes password with bcrypt (10 rounds), returns JWT
- `POST /api/login` — verifies password, returns JWT
- `GET /api/me` — validates token, returns username

### Game data

- `GET /api/profiles` — returns all profiles with `application_results` stripped (hidden from client)
- `GET /api/profiles/:id` — returns single complete profile including results (for Phase 4 reveal)

### Leaderboard

- `POST /api/scores` — saves score (requires JWT); computes best score per user per profile to prevent score inflation from replaying
- `GET /api/leaderboard` — returns top 100 players sorted by total points
- `GET /api/stats` — returns `profileCount`, `totalPlays`, `uniquePlayers`

### Database

- Uses `lowdb` (JSON file) instead of SQLite — no C++ compilation required, works on Node 24
  - JWT secret auto-generated and persisted across server restarts
- 

## 6. Data Pipeline (scripts/scrape.js) — New

- Fetches posts from r/collegeresults and r/ApplyingToCollege using Reddit's public JSON API (no login required)
- Sends each post to OpenRouter AI (Gemini Flash) with a structured extraction prompt
- Extracts: demographics, test scores, GPA, course rigor, ECs, awards, admission results, game metadata

- Saves extracted profiles to `data/queue.jsonl` for human review
  - `npm run approve` — interactive CLI to approve/reject/edit each queued profile before it goes live
  - Deduplication via `data/seen_ids.json` — never reprocesses the same Reddit post
- 

## 7. Files Unchanged from Original

- `phase1-profile.jsx` — profile display, no changes
- `phase2-tier.jsx` — tier selection, no changes
- `ranks.js` — rank ladder definitions, no changes
- `tiers.js` — school tier definitions, no changes
- `ui-primitives.jsx` — shared UI components, no changes
- `data.js` — kept for fallback if server unreachable